

On the logical (in)consistency of code-generating LLMs

Ke Dong ¹, William Hsu ¹, Pascal Hitzler ¹, and Eugene Y. Vasserman ¹

Abstract:

In recent years, generative Large Language Model (LLM)-based programming assistants have soared in popularity. Owing to the black-box nature of Deep Neural Networks on which they are based, there is ongoing concern about potential correctness issues in generated code. Recent work has analyzed the reliability of LLMs as code generators for a number of programming languages. Most of that work has examined syntactic correctness, but the logical consistency of syntactically-correct code is another critical factor affecting correctness. We test the performance of five light-weight LLMs on logical consistency tasks based on simple logical inversion, using real-world code samples from the CodeSearchNet Python dataset. Using Cohen’s d and Odds Ratio metrics, we show that there are significant differences in whether models, even those designed for code generation, produce “sensible” code, i.e., syntactically correct and also *logically consistent*. Moreover, the amount of context included in the prompt has a major effect on model performance: models perform *worse* when given comments and/or additional code around the function being tested. Overall, we find that models’ logical consistency is well-aligned with EvalPlus@1 scores (a popular measure of LLM code generation capability), and further confirm generative LLMs’ shortcomings in handling even the most simple of logical “reasoning” tasks, and challenges in handling contextual information.

Keywords: Large Language Models, Code Generation, Logic

1 Introduction

In recent years, generative Large Language Models (LLMs) have soared in popularity as programming assistants for engineers and researchers. Modern LLMs parameter sizes are measured in the tens to hundreds of billions, making them vastly more complex than traditional Deep Neural Networks. The increased complexity results in enhanced capability, but also raises reliability concerns [Na23]. Also, due to the confabulation/hallucination problem compounded by the black-box nature of LLMs, there are multiple potential issues with LLM-generated content [Ya23, Ta24a, ZG24] to (partially) mitigate these issues and improve the reliability of LLMs, many natural language (NL)-based studies focus on grammar-based analysis [Fi21]. Similarly, mainstream methods for enhancing the reliability of code generation LLMs are based on syntax analysis of the output [Sh22, NRR24, Ug24]. The information contained in the semantic structures has also received extensive attention, exploring LLMs’ inputs and outputs in more abstract and higher-dimensional contexts [Ta24b]. However,

¹ Kansas State University, Department of Computer Science, Manhattan, KS, 66505, United States,
keadriadong@ksu.edu,  <https://orcid.org/0009-0000-1087-1756>;
bhsu@ksu.edu,  <https://orcid.org/0000-0002-2950-6246>;
hitzler@ksu.edu,  <https://orcid.org/0000-0001-6192-3472>;
eyv@ksu.edu,  <https://orcid.org/0000-0002-2420-4329>

logical flaws, especially in code-generating LLMs’ output, have been less extensively explored [Fr23]. In this work, we study the logical correctness of LLM code generation using real-world Python code.

Several studies have already been conducted on logical relations and reasoning in NL [Wa24], as well as programming languages (PLs) [GCF24], but there is a lack of an intuitive and effective methodology to verify logical consistency in LLM-generated code. In this paper, we define a logical consistency check based on Hoare logic, which can formally represent the precondition and the postcondition of a specific piece of code. To apply the logical consistency check, we first find a representative pair of complementary Hoare logical triples, then derive the associated Python syntax-compliant triples. Based on the Python triples, we use Abstract Syntax Trees (ASTs) to filter the CodeSearchNet dataset [Hu19] and select the code samples satisfying our logical consistency requirements. Then, we perform Logical Inversion on each code sample and test the sensitivity of light-weight LLMs to logical consistency, including GPT-2 [Ra19], CodeGemma-2B [Te24], CodeGen-2B-Mono [Ri23], Phi-3-Mini-4K-Instruct [Bi24], and CodeLlama-7B [Ro23].

Our contributions are as follows:

- We propose Logical Consistency as a novel evaluation metric for an LLM’s code generation logic “ability”.
- We design a process to detect the sensitivity of LLMs to logical inversion as formally defined using Hoare Logic.
- We develop a code selection filter using Python’s `ast` module to detect real-world Python code examples satisfying the Hoare triples, and to create equivalent but logically inverted code from those samples.
- We evaluate LLMs code generation capability by performing Next Operation Prediction (NOP) and Fill-in-the-Middle (FIM) tasks for logical consistency detection using Cohen’s d and Odds Ratio metrics.

2 Related Work

Grammar and Syntax Analysis. Grammatical analysis of LLMs has been a hot topic in recent years. In 2021, Finlayson et al. examined the subject-predicate consistency performance in contexts of varying complexity to illustrate neuron behaviors inside language models [Fi21]. Causal analyses in multiple languages were used to investigate the sensitivity of language models to various fine-grained grammatical consistency (which refers to minor syntactic structures consisting of several consecutive words) [MXL22, Xi21, La22, SHK24]. The processing of fine-grained information by LLMs reflects both the sensitivity of an LLM to the specified grammatical structures and provides a quantitative metric for the syntactic consistency in LLM-generated content. Also, Tao et al. proposed a multi-level fine-grained detection (MFD) framework that integrates structural, semantic, and linguistic features,

assisting with the detection of LLM-generated content [Ta24b]. This study demonstrates LLMs' capacities to detect abstract syntactic information when processing grammatical structures with "word jumps", which refers to a phenomenon of sudden changes of the words in sentences during text generation that may lead to semantic or syntactic inconsistency. Transferred from NL scenarios, syntactic analysis of PLs using AST tools has been used to understand how LLMs generate code [Sh22, Zh24]. Meanwhile, parsing code structures using ASTs, e.g., assigning model confidence to AST nodes, has been shown to provide interpretability of code generation LLMs [Pa24]. Similarly to NL, LLMs are not only sensitive to specific syntactic structures but also generate code following implicit high-level restrictions, such as control flow graphs and context-free grammars [Hu23, Ug24].

Logical Analysis. Logical analysis is derived from high-level grammatical consistency. Due to the imprecision and ambiguity of NL, Friedman et al. found although LLMs can follow grammatical rules, they still fall short in "understanding" concepts and capturing logic [Fr23]. Xu et al. evaluated the logic limitation of LLMs from four perspectives: answer correctness, explanation correctness, explanation completeness, and explanation redundancy, and concluded all LLMs exhibit logical reasoning limitations, especially in induction tasks [Xu23]. LogicAsker introduced propositional logic into the analysis of NL, revealing the limitations of Chain-of-Thought (CoT) reasoning in certain tasks, and so emphasized the importance of reinforcing LLM foundational "reasoning" skills [Wa24]. As a formal language, PLs have stricter requirements for logic than NLs. Current work on logical analysis of LLM-generated code focuses on logical reasoning or formal interpretation. For example, Gandarela et al. combined LLMs with a formal inference engine to evaluate the capabilities and limitations of LLMs for logical theory induction by analyzing the reasoning performance of LLMs under different dependency structures through a complexity hierarchy methodology [GCF24]. Chen et al. proposed a framework called Dual Chains of Logic (DCoL), which employs dual-path reasoning to guide LLMs in simulating logic [Ch24]. Both studies above analyzed the weaknesses of the models and suggested introducing specific structures to improve the logic in code generated by LLMs. Overall, the focus of existing research on LLM-based logic analysis is mostly in two fields: enhancing LLMs' logic generation performance through various logical reasoning strategies, and evaluating the capacities of LLMs to understand logic by changing the ways they are prompted. We study how code generation LLMs respond to logical changes in the code with the same structure/syntax but logically inverted semantics – when the logical meaning of the original prompt is inverted, do LLM-generated predictions also invert accordingly? Our study fills a gap in existing research by testing whether LLMs can change predictions as we desired by introducing specific interventions in original code prompts to examine the logical consistency and robustness exhibited by LLMs in two practical programming tasks (NOP and FIM).

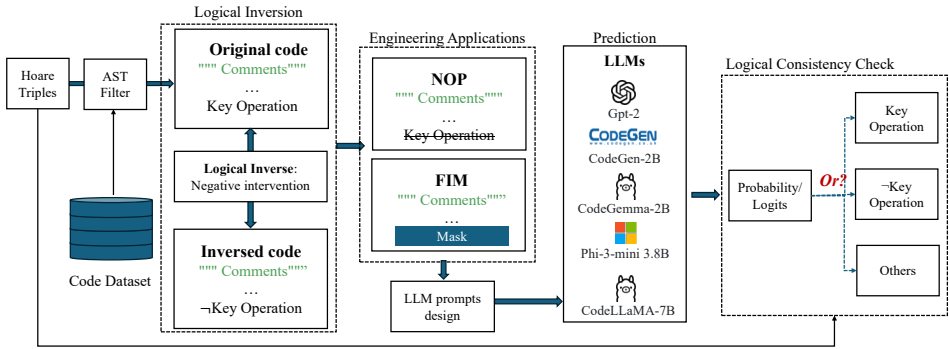


Fig. 1: LLM Logical Consistency Evaluation Methodology. We first designed Hoare Triples and then used an AST filter to extract code examples from the real Code Dataset. Next, we proposed Logical Inversion to logically inverse the Key Operation in the code. For engineering applications, two types of prompts, including Next Operation Prediction (NOP) and Fill-in-the-Middle (FIM), were designed for LLMs' predictions. The entire framework was evaluated through a Logical Consistency Check.

3 Methodology

This section outlines the formal foundations of our experiments and then lays out our framework for evaluating the logical consistency of code generated by LLMs, as depicted in Figure 1. Our code and results are available on GitHub.²

Hoare Triples. The standard Hoare triple is $\{P\}C\{Q\}$, which describes the current state of any program $\{P\}$, the transition operation C on that state, and the resulting state $\{Q\}$ after applying operation C . This can represent an arbitrary instruction, function, or a set of instructions or functions that moves program execution forward [Ho69, Re98]. In this work, we attempt to isolate the smallest “unit” of logical reasoning, and so investigate what we call “*Logical Inversion*”, a specific case where Q is the logical negation of P , i.e., the program’s execution of C results in the logical negation of the initial state, represented as $\{P\}C\{\neg P\}$.

The concept of duality [Wa03] represents the pair of propositions or operations that are structurally or logically complementary. We leverage this duality to ensure the robustness of a system by validating both a specific logical condition and its negated counterpart [Ge16] to confirm that the system maintains logical consistency in both scenarios. We can derive the Hoare triple pair that satisfies logical inversion as Equations (1) and (2):

$$\{P\} C \{\neg P\} \quad (1) \quad \{x \in L\} \text{ if } x \text{ in } L: L.\text{remove}(x) \{x \notin L\} \quad (3)$$

$$\{\neg P\} \neg C \{P\} \quad (2) \quad \{x \notin L\} \text{ if } x \text{ not in } L: L.\text{append}(x) \{x \in L\} \quad (4)$$

² <https://github.com/keadriadong/PL-LogicalConsistency>

The Hoare triple in Equation (1) represents the operation C which transforms the state P to its logical negation $\neg P$. The second Hoare triple in Equation (2) is the inverse of Equation (1), where the operation $\neg C$ transforms the state $\neg P$ back to P . The validation of dual triples ensures that the logical inversion is consistent and robust, providing a solid foundation for reasoning about logical operations in PLs.

In Python, we let operation C be either “append” or “remove”, depending on the precondition related to the membership of x in a list L . Here, P and $\neg P$ represent complementary conditions where P signifies $x \in L$ (i.e., x is a member of L), and $\neg P$ signifies $x \notin L$ (i.e., x is not a member of L). To instantiate concrete Python syntax, we decompose C into two steps: the first step tests whether or not x is in L and the second step (the *Key Operation*) inverts that condition. In theory, we could perform the *Key Operation* unconditionally, but in Python in particular, attempting to remove x from L when x is not in L would raise a `ValueError` exception, so the membership test is required. To avoid these runtime exceptions, Hoare Triples in Equations (1) and (2) require expansion into Equations (3) and (4), respectively.

Specification and Definition. We define “Logical Consistency” as follows: a test code snippet *has Logical Consistency* if a series of commands can successfully transform $x \in L$ to $x \notin L$ or vice versa. In Python, this transformation is achieved if the snippet first tests the membership of x in L using `In/NotIn`, followed by a *Key Operation* that inverts the membership status via `remove(x)/append(x)`. We utilize this definition to filter code examples and generate their dual counterparts for LLM prompt design. To ensure the logical relations between the primary elements, we impose the following constraints:

1. The logical relation must apply to the same x (variable or object), which limits the operations and reasoning scope to a specific x .
2. Logical relations must be direct and explicit; therefore, we limit *Key Operation* to a single operation, either `append` or `remove`.
3. Logical relations must exist within the same syntactic node and satisfy the `if:test→body` structure, a conditional test and an associated block of code (the body) that is executed when the condition evaluates to `True`, to avoid excessive information in the prompt.

These constraints are essential to eliminate syntactic features unrelated to logical inversion and clarify the code syntax structure we want. However, it does not mean that extra context is irrelevant, and we specifically design experiments that investigate the impact of redundant information in Section 4.1.

AST-based Logic Filter. Following Shen et al. [Sh22], we use the Python `ast` module to build the filter for systematically extracting Python code snippets satisfying either Equation (3) or (4), ensuring they align with the desired logical structures. Later, this module will also let us precisely identify and extract specific components (such as code comments). By transforming source code into a tree-structured representation, we can easily parse, process, and analyze the code structure at a granular level to automate the identification

and extraction of code snippets. (Note that we use ASTs only for code transformation, and prior work on LLMs and structured intermediate code representation to measure and correct model output is orthogonal to ours [Pa24, Hu23, Ug24].)

We use the `ast` module to build logical filters according to the following criteria:

1. The example contains `if:test→body` structures.
2. The test condition is a `compare` class and has a left value of type `Constant` (constant/number/characters) or `Name` (variable).
3. The comparison operator must be `In` or `NotIn`.
4. A `Call` node, wrapped by an `Expr` node, is present, and the called function has the attribute “`remove`” or “`append`”.
5. The args of the called function match the left value of the test condition.

By applying these criteria, we filter the CodeSearchNet dataset [Hu19] to collect real Python examples that match either Equations (3) or (4). Then, we apply logical inversion to each obtained Python example and end up with pairs of original (unmodified) code examples and their logically inverted counterparts.

Logical Inversion. Generating dual code pairs from a given code example is to create the complementary code by introducing a negative intervention. According to the criteria given above, the filtered code examples must have both a comparison operator `In` or `NotIn` and a *Key Operation* of either `remove` or `append` in a single `if:test→body` node. The introduction of negative intervention indicates that if a code example has `In` operator, we should inverse the `In` to `NotIn` and correspondingly adjust the `remove` to `append` in the body to keep the logic consistency, or vice versa.

We implement our intervention in Python using the `ast` module, we inherit the `NodeVisitor` and `NodeTransformer` base classes from Python `ast` module. `NodeVisitor` is employed to traverse the AST and identify `if:test→body` nodes that conform to the conditions laid out above. While `NodeVisitor` allows us to identify the relevant AST nodes and operations, we create their inverse by using the `NodeTransformer` class, which enables us to alter the structure of the input syntax tree. The process of generating dual code examples begins by comparing the left value of the *Key Operation* in the test condition with the argument of the function in the body. If the test condition’s left value matches the argument of the function in the body, we proceed to invert the logic by switching the operator (`In` to `NotIn` or vice versa) and adjusting the corresponding function (`remove` to `append` or vice versa).

This approach to generating dual examples allows us to explore how different logical structures are interpreted and processed by LLMs, providing valuable insights into the model’s handling of logically consistent code transformations.

4 Experiments

We design causal analysis experiments [Fi21] to test the relationships between LLM logical consistency and code generation capability. This section details our data collection and analysis activities, including a quantitative comparison of multiple light-weight LLMs’ code generation performance in terms of logical consistency.

4.1 Preliminaries

We use five “small” pre-trained LLMs with 7B or fewer parameters: GPT-2 [Ra19], CodeGemma-2B [Te24], CodeGen-2B-Mono [Ri23], Phi-3-Mini-4K-Instruct [Bi24], and CodeLlama-7B [Ro23]. Except GPT-2, all were specifically designed for code generation and fine-tuned on code datasets. Small-sized LLMs are often fine-tuned for differing specializations, e.g., CodeGemma-2B can do both code completion and infilling while CodeGen documentation discussed code generation but not infilling. GPT-2, CodeGemma-2B and CodeGen-2B-Mono were run locally using Python 3.10, PyTorch 2.2.1, and Transformer 4.38.2 on an Apple laptop with an M2 Max processor. Phi-3-Mini-4K-Instruct and CodeLlama-7B experiments were run on Google Colab Pro.²

EvalPlus@1. One of the most up-to-date metrics for LLM code generation capability is EvalPlus@1 [Li23], which builds upon HumanEval@1 [Ch21]. EvalPlus@1 measures the proportion of tasks for which the model generated the expected prediction in its first and only attempt on an augmented dataset with more diverse test cases. Generally, an LLM with a higher score is considered to have stronger code generation abilities. The EvalPlus@1 scores for models we test (except GPT-2, which is not scored) are: CodeGemma-2B (20.7), CodeGen-2B-Mono (22.6),³ Phi-3-Mini-4K-Instruct (59.1), and CodeLlama-7B (35.4) [Li].

Data Processing. We collect a dual logical inversion dataset by filtering the CodeSearchNet Python dataset [Hu19] using the AST-based filter proposed in Section 3 and the logical inversion in Section 3. The dual logical inversion dataset (Python code with structures that satisfied the Hoare Logic triple (Equations (3) and (4)) has 441 dual code pairs. The label of each code pair is the relative *Key Operation* in the original example. Among them, 140 code pairs are labeled with *remove* and 301 code pairs are labeled with *append*.

To evaluate the influence of additional contextual information, we process the code examples in three different ways to obtain the LLM prompts, that is, our logical inversion dataset is composed of three sub-datasets: *Original*, *No Comment*, and *No Context*. The **Original** dataset (see Figure 2a) is the complete code snippet with comments. Then, we built the **No Comment** dataset by removing all the comments in the **Original** code example pairs (see Figure 2c). The *No Context* dataset (see Figure 2e) is obtained by retaining only the basic logical framework of the code structures according to Equations (3) and (4).

³ There is no EvalPlus@1 score provided for the -Mono variant; this score is for CodeGen-2B

Note that the Original and No Comment experiments have a major limitation: because Python uses “duck typing”, we do not know whether L is a list or a set in the context of `if x in L`. This is important because both lists and sets have a `remove` operations, only lists have **append** (while sets have `add`). When inverting a condition, replacing `remove` with `append` unconditionally would cause an LLM prediction of `add` to be considered incorrect. This limitation can be overcome by testing languages with stronger type systems. It can be overcome in Python only if the data type (list or set) is created within the available code snippet. These experimental method refinements will be the subject of future work. The No Context experimental results are affected in a more limited way because no type information is available as the creation of the data structure, code comments, and function names are unavailable; only the names of the variables may hint at the data types.

Prompt Design for NOP and FIM tasks. We ran two different types of experiments on the logical inversion dataset: next operation prediction (NOP) and the Fill-in-Middle (FIM). The prompt design for *NOP* required that the code be truncated before the *Key Operation* so that the LLMs can directly predict the next operation. We can observe the effects of logical intervention by comparing LLMs’ probabilities in generating the specific candidates. In contrast to the direct truncation of the source code in *NOP*, *FIM* prompts are customized according to documentation for each LLM. The first step is to use a [MASK] symbol to hide the *Key Operation* and mark its location. Before prompting the LLM, we pre-process the prompt by replacing the [MASK] location with the LLM-specific infilling token. Figure 2 shows a real code snippet `get_themes` and the final prompt across the three sub-datasets (*Original*, *No Comment*, and *No Context*) for the two tasks (*NOP* and *FIM*). Note that we only report *FIM* results for CodeGemma-2B and CodeLlama-7B, which explicitly list code infilling capabilities in their official documentation. (While we performed FIM experiments with the remaining models (GPT-2, CodeGen-2B-Mono, and Phi-3-Mini-4K-Instruct), we omit these results as the models performed poorly, and we conjecture that they were not designed for FIM tasks.)

4.2 Comparative Experiments

In this section, we discuss the results of analyses using Cohen’s d and mean log Odds Ratios (OR) to compare the sensitivity of five selected LLMs to measure logical consistency. Extra token probability analysis is used to confirm that our choice of *Key Operation* is correct.

Cohen’s d . We measured the Cohen’s d effect value [FMR12] for each code example pair. Cohen’s d is a function of the mean and variance for each candidate separately before and after the intervention. We first collected LLMs’ prediction probabilities of both possible *Key Operations* before and after logical inversion, respectively. Then, the final Cohen’s d is calculated by the mean and variance of the obtained probabilities. In addition, a value of Cohen’s $d > 0$ indicates that the LLM-predicted probability of a specific *Key Operation* increases after logical inversion, and vice versa. Since the negative intervention is introduced, we can anticipate that: Given a ground truth label, the prediction probability of the same


```
def get_themes(templates_path):
    """Returns available themes list."""
    themes = os.listdir(templates_path)
    if '__common__' in themes:
        themes.
```

(a) The original NOP prompt

```
def get_themes(templates_path):
    themes = os.listdir(templates_path)
    if '__common__' in themes:
        themes.
```

(c) The No Comment NOP prompt

```
if '__common__' in themes:
    themes.
```

(e) The No Context NOP prompt

```
def get_themes(templates_path):
    """Returns available themes list."""
    themes = os.listdir(templates_path)
    if '__common__' in themes:
        themes.[MASK]('__common__')
    return themes
```

(b) The original FIM prompt

```
def get_themes(templates_path):
    themes = os.listdir(templates_path)
    if '__common__' in themes:
        themes.[MASK]('__common__')
    return themes
```

(d) The No Comment FIM prompt

```
if '__common__' in themes:
    themes.[MASK]('__common__')
```

(f) The No Context FIM prompt

Fig. 2: The prompts for completion (NOP) and fill-in-the-middle (FIM) for the get_themes example

Key Operation with the label will decrease after the intervention introduced, while the complementary *Key Operation* is more likely to be predicted. If the result of Cohen’s d aligns with our expectations, we refer to the overall changes as the specific LLM having sensitivity to logical consistency. The absolute value of Cohen’s d indicates the extent of sensitivity, that is, the overall probability change after introducing the logical inversion. The results are shown in Tables 2 and 3.

In the NOP task, results demonstrate that LLMs are more likely to exhibit logical consistency in the No Context scenario – when the code is stripped of all surrounding code and comments. Although GPT-2 does not exhibit logical consistency in Original and No Comment, it shows some sensitivity when dealing with No Context data. CodeGemma-2B, CodeGen-2B, Phi-3-Mini-4K-Instruct, and CodeLlama-7B were relatively successful in logical consistency tests on all three sub-datasets, but showed different sensitivities. Similarly, both CodeGemma-2B and CodeLlama-7B are sensitive to logical consistency when predicting the masked *Key Operation*. Overall, LLMs show better logical consistency performance on the No Context sub-dataset compared to the sub-datasets that contain redundant/contextual information.

To validate the reliability of Cohen’s d , we used the Kolmogorov-Smirnov test to assess whether the probabilities of *Key Operation* before logical inversion followed a normal distribution. As expected, the LLMs’ probability predictions deviated from normality. While Cohen’s d is still effective for reflecting the overall degree of intervention, even for non-normal data [AK03, He81], we introduce Odds Ratio (OR) as a complementary and more precise metric to measure the effect of logical inversion on LLMs predictions.

		Original		No Comment		No Context	
		remove	append	remove	append	remove	append
GPT-2	remove	0.0572	-0.2513	0.1431	-0.306	0.2334	-0.2402
	append	0.0954	-0.0815	0.0904	-0.0893	-0.0203	0.0229
CodeGemma-2B	remove	-2.1004	1.9963	-3.0742	2.0518	-2.3209	1.7656
	append	1.6453	-1.9521	2.2528	-2.0298	2.7048	-2.1337
CodeGen-2B-Mono	remove	-2.2898	1.9595	-2.7335	1.9505	-3.8425	2.2782
	append	1.7148	-1.7465	2.0988	-1.7643	2.7367	-2.2126
Phi-3-Mini-4K-Instruct	remove	-1.2804	2.0417	-1.6463	2.0496	-3.588	2.2588
	append	1.0159	-2.0042	1.3066	-1.9814	2.8616	-2.1579
CodeLlama-7B	remove	-2.0472	2.0746	-2.4921	2.1593	-4.3375	2.3515
	append	1.6509	-2.008	1.9661	-2.0746	3.2095	-2.5095

Tab. 2: Cohen’s d for NOP task. The sub-columns represent the ground truth labels and the sub-rows are the LLM-predicted *Key Operations*. **Bold** indicates particular *Key Operation* is most affected overall by intervention, which is consistent with our expectation of logical consistency.

Odds Ratio (OR). Similar to Wan et al. [Wa23], we calculate logarithmic Odds Ratio (OR) to assess the occurrence rate changes of the label (*Key Operation*) for each code example *before and after* the negative intervention introduced (seen in Section 3) in the NOP and FIM tasks. Negative OR implies the occurrence rates of the label after the intervention is decreased, which shows the model logically inverts its predictions in response to logical inversion, and vice versa. We follow Grimes et al. to calculate the original OR [GS08], and use a natural logarithm to amplify the OR value:

$$\text{OR} = \ln \left(\frac{\text{odds}_1}{\text{odds}_0} \right) = \ln \left(\frac{p_1/(1-p_1)}{p_0/(1-p_0)} \right) \quad (5)$$

where p_0 is the probability of label function before intervention and p_1 is the probability after intervention. We compute the mean OR separately for ORs of the individual example with the same labels in each sub-dataset; the results of mean OR are in Tables 4 and 5.

		Original		No Comment		No Context	
		remove	append	remove	append	remove	append
CodeGemma-2B	remove	-1.3406	1.0157	-1.7518	1.2996	-3.6853	3.2387
	append	1.9601	-1.5009	2.0604	-1.5064	2.1267	-2.286
CodeLlama-7B	remove	-1.4273	1.3026	-1.7167	1.548	-4.1308	3.1596
	append	2.1862	-2.2778	2.279	-2.3382	2.5303	-2.5134

Tab. 3: Cohen’s d for FIM task. The sub-columns represent the ground truth labels and the sub-rows are the LLM-predicted *Key Operations*. **Bold** indicates that the LLM’s prediction of a particular *Key Operation* is most affected overall by intervention, which is consistent with our expectations. (We omit models without explicit FIM instructions, but the results are available on our GitHub page.²)

	Original		No Comment		No Context	
	remove	append	remove	append	remove	append
GPT-2	0.2125	-0.2008	0.2921	-0.219	0.2727	0.0205
CodeGemma-2B	-4.3629	-4.469	-5.504	-4.6644	-4.947	-2.6455
CodeGen-2B-Mono	-5.223	-5.2949	-5.4208	-4.8922	-7.1689	-3.8267
Phi-3-Mini-4K-Instruct	-4.3554	-5.5964	-5.0424	-5.2572	-8.4055	-3.9519
CodeLlama-7B	-5.156	-5.0494	-5.6509	-4.9857	-8.0898	-3.8988

Tab. 4: Mean OR for NOP task. The sub-columns are the ground truth (content of the original code snippet). **Bold** indicates that the LLM shows the strongest sensitivity to logical consistency.

The results show that most LLMs of both tasks show logical consistency, except GPT-2. Especially, Phi-3-Mini-4K-Instruct achieves excellent logical consistency in NOP, surpassing CodeLlama-7B, which has more parameters, consistent with EvalPlus@1 scores. Similarly, CodeLlama-7B outperforms CodeGemma-2B in FIM, which is also consistent with their EvalPlus@1 scores. Note that both CodeGemma-2B and CodeLlama-7B show different capacities to detect logical consistency in NOP and FIM and different sensitivities to different logical consistency structures. For example, CodeLlama-7B is more sensitive to the In-remove structure across three sub-datasets in NOP, while the NotIn-append structure is better detected in FIM. CodeGemma-2B demonstrated similar trends in logical consistency across the two tasks; its performance in NOP is more stable, but the OR is significantly high in the No Context dataset in FIM. It is not yet clear how these features relate to the fine-tuning of the models on the purpose of code infilling.

In addition, we observe that Cohen’s d and OR give different results in assessing the LLMs’ sensitivity to logical consistency. For example, in Table 2, CodeLlama-7B “seems” to perform better than Phi-3-Mini-4K-Instruct. This is because Cohen’s d focuses on the average difference between the two group of values, and therefore can only suggest that the probability distributions of CodeLlama-7B differ considerably before and after intervention. On the other hand, OR focuses on the relative odds of a single event occurrence, and thus better reflects the strength of logical consistency. Therefore, we can use Cohen d to measure

	Original		No Comment		No Context	
	remove	append	remove	append	remove	append
CodeGemma-2B	-3.2309	-3.4589	-3.8635	-3.4555	-7.7356	-3.2847
CodeLlama-7B	-3.3176	-5.6244	-3.8665	-5.4911	-6.8403	-3.8399

Tab. 5: Mean OR for FIM task; the sub-columns are the ground truth. **Bold** indicates that the LLM shows the strongest sensitivity to logical consistency. (We omit models without explicit FIM instructions, but the results are available on our GitHub page.²)

the overall sensitivity of LLMs to logical consistency under a label-candidate pair, but OR is a better indicator of the specific behavior of the model for particular data. The sensitivity of LLMs to logical consistency is aligned with their EvalPlus@1 scores.

5 Discussion

Our experimental results show a clear and important shortcoming of code generation LLMs’ “reasoning” performance for Python list/set membership tasks based on a small “unit” of logical reasoning. We expect that these results generalize to other languages and other simple logical structures, and (dis)confirming our hypothesis is a subject for future work.

Our research takes a step toward testing the logical consistency of LLMs in code generation tasks, provides a more understandable measurement of real-world code generation tasks, and offers a potentially more complete solution to LLM-generated code correctness evaluation. Our results are aligned with existing metrics to evaluate LLM code generation in that LLMs with higher EvalPlus@1 scores tend to have stronger sensitivity to logical consistency. We evaluated two types of the most common code completion tasks for logical consistency: operation prediction (NOP) and the Fill-in-Middle (FIM). In experiments across five light-weight LLMs including GPT-2, CodeGemma-2B, CodeGen-2B-Mono, Phi-3-Mini-4K-Instruct, and CodeLlama-7B; we found Phi-3-Mini-4K-Instruct is the most sensitive to logical consistency while also achieving the highest EvalPlus@1 score (59.1) of all the evaluated models. Since we did not find official documentation for constructing infilling prompts for GPT-2, CodeGen-2B-Mono, or Phi-3-Mini-4K-Instruct, we only compared CodeGemma-2B and CodeLlama-7B in FIM; the results show that CodeLlama-7B (EvalPlus@1=35.4) outperforms CodeGemma-2B (EvalPlus@1=20.7). Note that the performance of CodeGemma-2B and CodeLlama-7B is different in NOP and FIM tasks. On the Original and No Comment sub-datasets, both CodeGemma-2B and CodeLlama-7B perform better in the NOP task by achieving a higher absolute log OR. However, the FIM performance of CodeGemma-2B in the No Context scenario is better than its NOP performance, consistent with the prompts for FIM containing more contextual information (specifically, the argument x for the *Key Operation*). In other datasets, additional context may lead to unstable performance. For instance, CodeLlama-7B performs better in the FIM task than the NOP task in the No Context scenario. Thus, performance may show more complex dependence on model training and fine-tuning strategies rather than simply on the amount or nature of the context. Studying these detailed differences is a topic for future work. We also ran supplementary experiments to eliminate the ambiguity in prediction accuracy measures due to logical “synonyms” inherent to Python’s ambiguous (duck typing) system. These results are omitted for lack of space, but show only minor interference, except in GPT-2 (which was not designed for code completion). The additional results do hint at the number of potential false negatives (LLM predictions deemed incorrect when they may have been correct) is available by examining the frequency of add vs. append predictions. The code and associated results can be found in our code repository.²

Bibliography

- [AK03] Algina, James; Keselman, HJ: Approximate confidence intervals for effect sizes. *Educational and Psychological Measurement*, 63(4):537–553, 2003.
- [Bi24] Bilenko, Misha: Introducing Phi-3: Redefining what’s possible with SLMs. <https://azure.microsoft.com/en-us/blog/introducing-phi-3-redefining-whats-possible-with-slms/>, April 2024. Available at <https://ai.azure.com/explore/models/Phi-3-mini-4k-instruct/version/13/registry/azureml>.
- [Ch21] Chen, Mark; Tworek, Jerry; Jun, Heewoo; Yuan, Qiming; Pinto, Henrique Ponde De Oliveira; Kaplan, Jared; Edwards, Harri; Burda, Yuri; Joseph, Nicholas; Brockman, Greg et al.: Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.
- [Ch24] Chen, Minyu; Li, Guoqiang; Wu, Ling-I; Liu, Ruibang; Su, Yuxin; Chang, Xi; Xue, Jianxin: Can Language Models Pretend Solvers? Logic Code Simulation with LLMs. *arXiv:2403.16097*, 2024.
- [Fi21] Finlayson, Matthew; Mueller, Aaron; Gehrmann, Sebastian; Shieber, Stuart; Linzen, Tal; Belinkov, Yonatan: Causal analysis of syntactic agreement mechanisms in neural language models. In: Joint Conference of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL-IJCNLP 2021. Association for Computational Linguistics (ACL), pp. 1828–1843, 2021.
- [FMR12] Fritz, Catherine O; Morris, Peter E; Richler, Jennifer J: Effect size estimates: Current use, calculations, and interpretation. *Journal of experimental psych.: General*, 141(1):2, 2012.
- [Fr23] Friedman, Robert: Large language models and logical reasoning. *Encyclopedia*, 3(2):687–697, 2023.
- [GCF24] Gandarela, João Pedro; Carvalho, Danilo S; Freitas, André: Inductive Learning of Logical Theories with LLMs: A Complexity-graded Analysis. *arXiv:2408.16779*, 2024.
- [Ge16] Gehrke, Mai: Duality in computer science. In: *ACM/IEEE Symposium on Logic in Computer Science*. pp. 12–26, 2016.
- [GS08] Grimes, David A.; Schulz, Kenneth F.: Making sense of odds and odds ratios. *Obstetrics & Gynecology*, 111(2 Part 1):423–426, 2008.
- [He81] Hedges, Larry V: Distribution theory for Glass’s estimator of effect size and related estimators. *Journal of Educational Statistics*, 6(2):107–128, 1981.
- [Ho69] Hoare, Charles Antony Richard: An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [Hu19] Husain, Hamel; Wu, Ho-Hsiang; Gazit, Tiferet; Allamanis, Miltiadis; Brockschmidt, Marc: Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv:1909.09436*, 2019.
- [Hu23] Huang, Qing; Zou, Zhou; Xing, Zhenchang; Zuo, Zhenkang; Xu, Xiwei; Lu, Qinghua: AI chain on large language model for unsupervised control flow graph generation for statically-typed partial code. *arXiv:2306.00757*, 2023.

- [La22] Lakretz, Yair; Desbordes, Théo; Hupkes, Dieuwke; Dehaene, Stanislas: Can transformers process recursive nested constructions, like humans? In: *Proceedings of the 29th International Conference on Computational Linguistics*. pp. 3226–3232, 2022.
- [Li] Liu, Jiawei; Xia, Chunqiu Steven; Wang, Yuyao; Zhang, Lingming: EvalPlus Leaderboard. <https://evalplus.github.io/leaderboard.html>. Accessed 31-10-2024.
- [Li23] Liu, Jiawei; Xia, Chunqiu Steven; Wang, Yuyao; Zhang, Lingming: Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In: *Advances in Neural Information Processing Systems (NeurIPS)*. volume 36, pp. 21558–21572, 2023.
- [MXL22] Mueller, Aaron; Xia, Yu; Linzen, Tal: Causal Analysis of Syntactic Agreement Neurons in Multilingual Language Models. In: *26th Conference on Computational Natural Language Learning, CoNLL 2022 collocated and co-organized with EMNLP 2022*. Association for Computational Linguistics (ACL), pp. 95–109, 2022.
- [Na23] Naveed, Humza; Khan, Asad Ullah; Qiu, Shi; Saqib, Muhammad; Anwar, Saeed; Usman, Muhammad; Akhtar, Naveed; Barnes, Nick; Mian, Ajmal: A comprehensive overview of large language models. *arXiv:2307.06435*, 2023.
- [NRR24] Netz, Lukas; Reimer, Jan; Rumpe, Bernhard: Using Grammar Masking to Ensure Syntactic Validity in LLM-based Modeling Tasks. In: *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*. pp. 115–122, 2024.
- [Pa24] Palacio, David N; Rodriguez-Cardenas, Daniel; Velasco, Alejandro; Khati, Dipin; Moran, Kevin; Poshyanyk, Denys: Towards More Trustworthy and Interpretable LLMs for Code through Syntax-Grounded Explanations. *arXiv:2407.08983*, 2024.
- [Ra19] Radford, Alec; Wu, Jeffrey; Child, Rewon; Luan, David; Amodei, Dario; Sutskever, Ilya et al.: Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [Re98] Reynolds, John C: *Theories of programming languages*. Cambridge University Press, 1998.
- [Ri23] Rivas, Alba: Inside CodeGen, Our In-House Open-Source LLM. <https://developer.salesforce.com/blogs/2023/11/inside-codegen-our-in-house-open-source-llm>, November 2023.
- [Ro23] Roziere, Baptiste; Gehring, Jonas; Gloeckle, Fabian; Sootla, Sten; Gat, Itai; Tan, Xiaoqing Ellen; Adi, Yossi; Liu, Jingyu; Sauvestre, Romain; Remez, Tal et al.: Code Llama: Open foundation models for code. *arXiv:2308.12950*, 2023.
- [Sh22] Shen, Da; Chen, Xinyun; Wang, Chenguang; Sen, Koushik; Song, Dawn: Benchmarking language models for code syntax understanding. *Findings of the Association for Computational Linguistics (EMNLP)*, pp. 3071–3093, December 2022.
- [SHK24] Sukumaran, Priyanka; Houghton, Conor; Kazanina, Nina: Investigating grammatical abstraction in language models using few-shot learning of novel noun gender. In: *Findings of the Association for Computational Linguistics: EACL 2024*. pp. 747–765, 2024.
- [Ta24a] Tang, Yihong; Wang, Bo; Wang, Xu; Zhao, Dongming; Liu, Jing; Zhang, Jijun; He, Ruifang; Hou, Yuexian: RoleBreak: Character Hallucination as a Jailbreak Attack in Role-Playing Systems. *arXiv:2409.16727*, 2024.

- [Ta24b] Tao, Zhen; Li, Zhiyu; Chen, Runyu; Xi, Dinghao; Xu, Wei: Unveiling Large Language Models Generated Texts: A Multi-Level Fine-Grained Detection Framework. *arXiv:2410.14231*, 2024.
- [Te24] Team, CodeGemma: CodeGemma: Open code models based on Gemma. *arXiv:2406.11409*, 2024.
- [Ug24] Ugare, Shubham; Suresh, Tarun; Kang, Hangoo; Misailovic, Sasa; Singh, Gagandeep: Improving LLM code generation with grammar augmentation. *arXiv:2403.01632*, 2024.
- [Wa03] Wadler, Philip: Call-by-value is dual to call-by-name. In: *ACM SIGPLAN conference on Functional Programming*. pp. 189–201, 2003.
- [Wa23] Wan, Yixin; Pu, George; Sun, Jiao; Garimella, Aparna; Chang, Kai-Wei; Peng, Nanyun: “Kelly is a Warm Person, Joseph is a Role Model”: Gender Biases in LLM-Generated Reference Letters. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. pp. 3730–3748, 2023.
- [Wa24] Wan, Yuxuan; Wang, Wenxuan; Yang, Yiliu; Yuan, Youliang; Huang, Jen-tse; He, Pinjia; Jiao, Wenxiang; Lyu, Michael R: LogicAsker: Evaluating and Improving the Logical Reasoning Ability of Large Language Models. In: *EMNLP*. 2024.
- [Xi21] Xiang, Beilei; Yang, Changbing; Li, Yu; Warstadt, Alex; Kann, Katharina: CLiMP: A Benchmark for Chinese Language Model Evaluation. In: *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. pp. 2784–2790, 2021.
- [Xu23] Xu, Fangzhi; Lin, Qika; Han, Jiawei; Zhao, Tianzhe; Liu, Jun; Cambria, Erik: Are large language models really good logical reasoners? A comprehensive evaluation from deductive, inductive and abductive views. *arXiv:2306.09841*, 2023.
- [Ya23] Yao, Jia-Yu; Ning, Kun-Peng; Liu, Zhen-Hui; Ning, Mu-Nan; Yuan, Li: LLM lies: Hallucinations are not bugs, but features as adversarial examples. *arXiv:2310.01469*, 2023.
- [ZG24] Zablocki, Piotr; Gajewska, Zofia: Assessing hallucination risks in large language models through internal state analysis. *Authorea Preprints*, 2024.
- [Zh24] Zhang, Quanjun; Shang, Ye; Fang, Chunrong; Gu, Siqi; Zhou, Jianyi; Chen, Zhenyu: TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models. *arXiv:2409.17561*, 2024.